

字符编码

从按下键盘，到屏幕出现汉字

同一份文字，为什么会出现乱码？

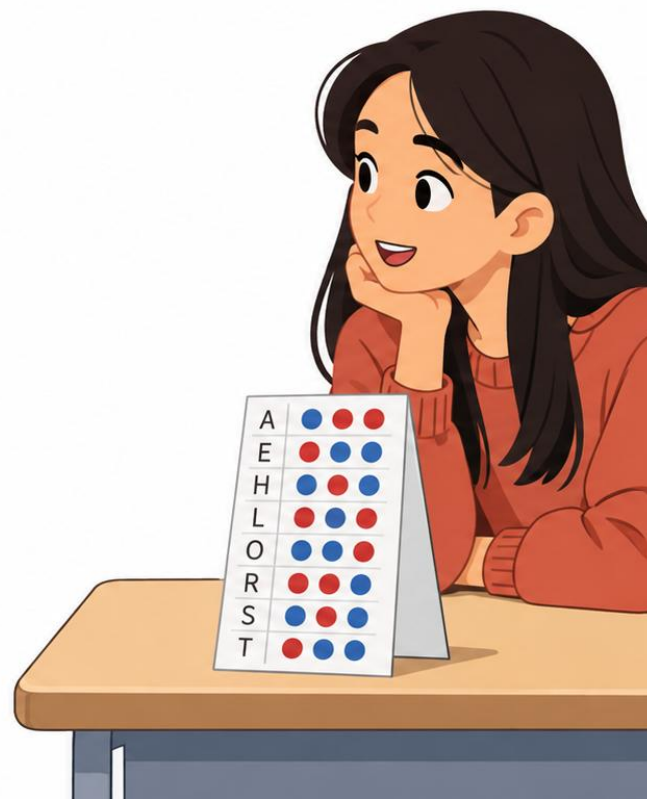
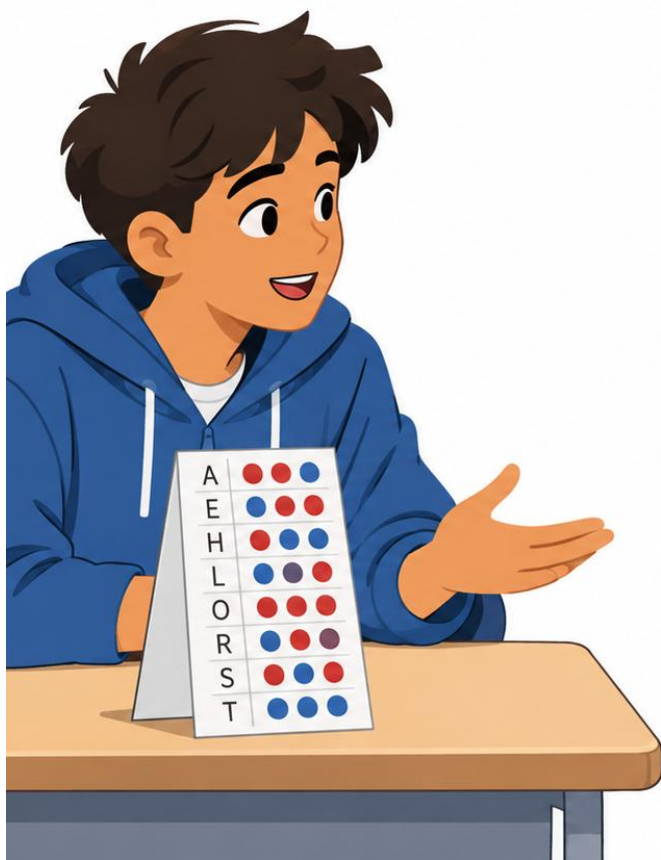
正常显示

你好

错误显示

浣豺ソ

玩个游戏：只用 3 bit, 发明一套编码



玩个游戏：只用 3 bit，发明一套编码

字符集 = {A, E, H, L, O, R, S, T}

A	E	H	L	O	R	S	T
000	001	010	011	100	101	110	111

环节 1：建立自己的秘密映射表

- 1) 拿出一张白纸
- 2) 把 000 到 111 **打乱**后分给这些字母，并在纸上记录
- 3) 不要给别人看自己设计编码表

玩个游戏：只用 3 bit，发明一套编码

字符集 = {A, E, H, L, O, R, S, T}

A	E	H	L	O	R	S	T
000	001	010	011	100	101	110	111

环节 2:

- 1) 拿出另一张白纸
 - 2) 用自己设计的编码表编码 **HELLO**，记录bit串在另一张白纸上
 - 3) 将bit串给你旁边的同学，看看对方能解读出什么
- 注：只给 bit串，不给对方你的编码表

玩个游戏：只用 3 bit，发明一套编码

字符集 = {A, E, H, L, O, R, S, T}

A	E	H	L	O	R	S	T
000	001	010	011	100	101	110	111

环节 3:

- 1) 彼此交换编码表
- 2) 结合编码表，再次解读bit串

一串 bits 天生具有文字意义吗？

同一串 bits

010 001 011 011 100



自己的表 → 可能读错

发送者的表 → 恢复 HELLO

bits 本身没有文字意义；共享映射规则后才有确定解释。

如果要同时编码HELLO和START，但只有 8 个位置， 哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO

STAR

如果要同时编码HELLO和START，但只有 8 个位置， 哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO $\rightarrow$ {H, E, L, O}

STAR $\rightarrow$ {S, T, A, R}

合并 $\rightarrow$ {A, E, H, L, O, R, S, T}

如果要同时编码HELLO和START，但只有 8 个位置， 哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO $\rightarrow$ {H, E, L, O}

STAR $\rightarrow$ {S, T, A, R}

合并 $\rightarrow$ {A, E, H, L, O, R, S, T}

CODE

如果要同时编码HELLO和START，但只有 8 个位置， 哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → {H, E, L, O}

STAR → {S, T, A, R}

合并 → {A, E, H, L, O, R, S, T}

CODE

C、D 不在字符集中

如果要同时编码HELLO和START，但只有 8 个位置， 哪些字符能进入字符集？

候选字符：A C D E H L O R S T X Y

HELLO → {H, E, L, O}

STAR → {S, T, A, R}

合并 → {A, E, H, L, O, R, S, T}

CODE

C、D 不在字符集中

先确定字符集，再为字符建立唯一的编码映射。

活动：请你读一读ASCII 表？

B \ b7 ----->					0	0	0	0	1	1	1	1
I \ b6 ----->					0	0	1	1	0	0	1	1
T \ b5 ----->					0	1	0	1	0	1	0	1
S												
COLUMN->					0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	0	_	o	DEL

- 1) 有哪些字符?
- 2) 字符A 的编码是什么?
- 3) 一共有多少个字符?

B \ b7 ----->					0	0	0	0	1	1	1	1
I \ b6 ----->					0	0	1	1	0	0	1	1
T \ b5 ----->					0	1	0	1	0	1	0	1
S												
COLUMN->					0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	0	_	o	DEL

1) 有哪些字符?

ASCII里，为什么有些“字符”看不见？



看不见 ≠ 没有意义

BEL 响铃

CR 回车

LF 换行

控制字符说明：早期字符编码服务的不只是“显示文字”，也服务于通信与设备控制。

B \ b7 ----->					0	0	0	0	1	1	1	1
I \ b6 ----->					0	0	1	1	0	0	1	1
T \ b5 ----->					0	1	0	1	0	1	0	1
S												
COLUMN->					0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	0	_	o	DEL

2) 字符A 的编码是什么?

B \ b7	0	0	0	0	1	1	1	1
I \ b6	0	0	1	1	0	0	1	1
T \ b5	0	1	0	1	0	1	0	1
S								
COLUMN→	0	1	2	3	4	5	6	7
b4 b3 b2 b1 ROW								
0 0 0 0 0	NUL	DLE	SP	0	@	P	`	p
0 0 0 1 1	SOH	DC1	!	1	A	Q	a	q
0 0 1 0 2	STX	DC2	"	2	B	R	b	r
0 0 1 1 3	ETX	DC3	#	3	C	S	c	s
0 1 0 0 4	EOT	DC4	\$	4	D	T	d	t
0 1 0 1 5	ENQ	NAK	%	5	E	U	e	u
0 1 1 0 6	ACK	SYN	&	6	F	V	f	v
0 1 1 1 7	BEL	ETB	'	7	G	W	g	w
1 0 0 0 8	BS	CAN	(	8	H	X	h	x
1 0 0 1 9	HT	EM	)	9	I	Y	i	y
1 0 1 0 10	LF	SUB	*	:	J	Z	j	z
1 0 1 1 11	VT	ESC	+	;	K	[	k	{
1 1 0 0 12	FF	FS	,	<	L	\	l	
1 1 0 1 13	CR	GS	-	=	M	]	m	}
1 1 1 0 14	SO	RS	.	>	N	^	n	~
1 1 1 1 15	SI	US	/	?	O	_	o	DEL

2) 字符A 的编码是什么? 1000001

```
(base) → repo ascii
Usage: ascii [-adxohv] [-t] [char-alias...]
-t = one-line output  -a = vertical format
-d = decimal table  -o = octal table  -x = hex table  -b binary table
-h = this help screen -v = version information
Prints all aliases of an ASCII character. Args may be chars, C \-escapes,
English names, ^-escapes, ASCII mnemonics, or numerics in decimal/octal/hex.
```

Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		Dec	Hex		
0	00	NUL	16	10	DLE	32	20		48	30	0	64	40	@	80	50	P	96	60	`	
1	01	SOH	17	11	DC1	33	21	!	49	31	1	65	41	A	81	51	Q	97	61	a	
2	02	STX	18	12	DC2	34	22	"	50	32	2	66	42	B	82	52	R	98	62	b	
3	03	ETX	19	13	DC3	35	23	#	51	33	3	67	43	C	83	53	S	99	63	c	
4	04	EOT	20	14	DC4	36	24	\$	52	34	4	68	44	D	84	54	T	100	64	d	
5	05	ENQ	21	15	NAK	37	25	%	53	35	5	69	45	E	85	55	U	101	65	e	
6	06	ACK	22	16	SYN	38	26	&	54	36	6	70	46	F	86	56	V	102	66	f	
7	07	BEL	23	17	ETB	39	27	'	55	37	7	71	47	G	87	57	W	103	67	g	
8	08	BS	24	18	CAN	40	28	(	56	38	8	72	48	H	88	58	X	104	68	h	
9	09	HT	25	19	EM	41	29	)	57	39	9	73	49	I	89	59	Y	105	69	i	
10	0A	LF	26	1A	SUB	42	2A	*	58	3A	:	74	4A	J	90	5A	Z	106	6A	j	
11	0B	VT	27	1B	ESC	43	2B	+	59	3B	;	75	4B	K	91	5B	[	107	6B	k	
12	0C	FF	28	1C	FS	44	2C	,	60	3C	<	76	4C	L	92	5C	\	108	6C	l	
13	0D	CR	29	1D	GS	45	2D	-	61	3D	=	77	4D	M	93	5D	]	109	6D	m	
14	0E	SO	30	1E	RS	46	2E	.	62	3E	>	78	4E	N	94	5E	^	110	6E	n	
15	0F	SI	31	1F	US	47	2F	/	63	3F	?	79	4F	O	95	5F	_	111	6F	o	
																			112	70	p
																			113	71	q
																			114	72	r
																			115	73	s
																			116	74	t
																			117	75	u
																			118	76	v
																			119	77	w
																			120	78	x
																			121	79	y
																			122	7A	z
																			123	7B	{
																			124	7C	
																			125	7D	}
																			126	7E	~
																			127	7F	DEL

B \ b7 ----->					0	0	0	0	1	1	1	1
I \ b6 ----->					0	0	1	1	0	0	1	1
T \ b5 ----->					0	1	0	1	0	1	0	1
S												
COLUMN->					0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	0	_	o	DEL

3) 一共有多少个字符?

B \ b7 ----->					0	0	0	0	1	1	1	1
I \ b6 ----->					0	0	1	1	0	0	1	1
T \ b5 ----->					0	1	0	1	0	1	0	1
S												
COLUMN->					0	1	2	3	4	5	6	7
b4	b3	b2	b1	ROW								
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	-	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	0	_	o	DEL

3) 一共有多少个字符? $2^7 = 128$

ASCII 表怎样读？

把两列拼成 7-bit 值：A、a、0、9、Space、LF。

字符	高 3 位	低 4 位
LF	000	1010
Space	010	0000
0	011	0000
9	011	1001
A	100	0001
a	110	0001

ASCII 表怎样读？

把两列拼成 7-bit 值：A、a、0、9、Space、LF。

字符	高 3 位	低 4 位
LF	000	1010
Space	010	0000
0	011	0000
9	011	1001
A	100	0001
a	110	0001

A: $100 + 0001 = 1000001_2 = 0x41 = 65$

ASCII 是 7 bit, 为什么常看到 8 bit?

ASCII 是 7 bit, 为什么常看到 8 bit?

Character:	A
Decimal code:	65
Hex code:	0x41
ASCII 7-bit code :	1000001
Stored in 1 byte :	01000001
MSB:	0

7-bit ASCII

1 0 0 0 0 0 1



存入 1 byte

0 1 0 0 0 0 0 1

ASCII 的编码值没变；只是用 8-bit byte 存放。



数字、大写、小写在 ASCII 中谁更小？

数字、大写、小写在 ASCII 中谁更小？

数字字符

'0' 0x30
↓
'9' 0x39

大写字母

'A' 0x41
↓
'Z' 0x5A

小写字母

'a' 0x61
↓
'z' 0x7A

编码值顺序：数字字符 < 大写字母 < 小写字母。

如何设计汉字编码？

如何设计汉字编码？ASCII 的哪些思想还能用？

如何设计汉字编码？ASCII 的哪些思想还能用？

ASCII 给出的设计问题

先确定字符集

每个字符唯一映射

ASCII 有 128 个位置

常用 1 byte 存放

已有 ASCII 文本

如何设计汉字编码？ASCII 的哪些思想还能用？

ASCII 给出的设计问题

先确定字符集



每个字符唯一映射



ASCII 有 128 个位置



常用 1 byte 存放



已有 ASCII 文本



汉字编码必须重新回答

哪些汉字进入字符集？

每个汉字对应什么编码？

容量够吗？

1 byte 够吗？

怎样兼容已有文本？

字符集与映射可以借鉴；容量与存储必须重新设计。

汉字写进文件以后是什么？

汉字写进文件以后是什么？

样本：A中B国C文

41	D6 D0	42	B9 FA	43	CE C4
A	中	B	国	C	文

“中” → D6 D0

ASCII 字符
1 byte

GB2312 样本汉字
2 bytes

D6 D0 为什么能代表“中”？

D6 D0 为什么能代表“中”？

“中”	国标码	56	50
		+80	+80
	机内码	D6	D0

MSB 区分

ASCII
0xxxxxxx

汉字
1xxxxxxx 1xxxxxxx

这里的“机内码”只限于 GB2312 双字节模型。

汉字真的总是占 2 bytes 吗？

汉字真的总是占 2 bytes 吗？

同一个字符：“中”

GB2312
D6 D0
2 bytes



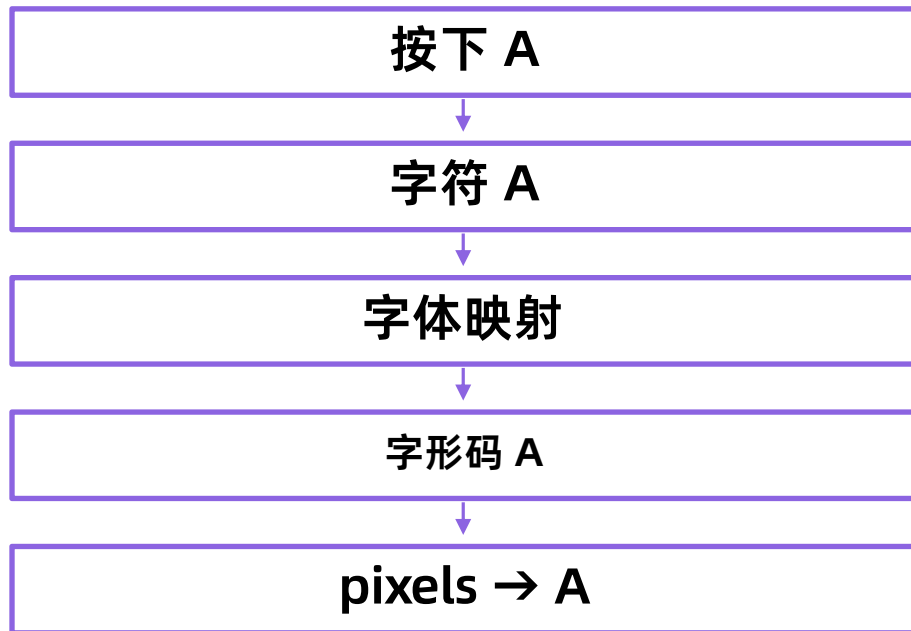
UTF-8
E4 B8 AD
3 bytes

“汉字占 2 bytes” 不是汉字本身的属性。

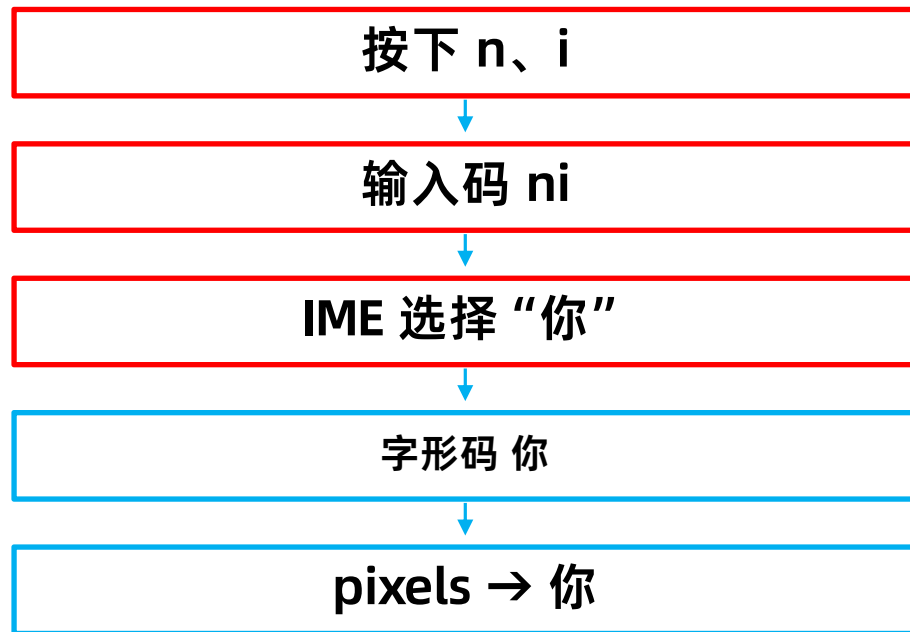
英文字符和汉字，从键盘到屏幕有什么不同？

英文字符和汉字，从键盘到屏幕有什么不同？

英文字符 A



中文字符“你”

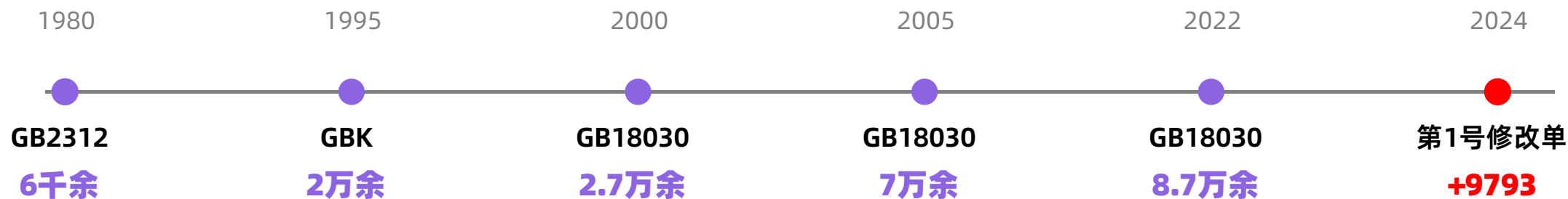


中文输入多了“输入码 → IME → 选字”；
显示阶段仍要经过 字形码 → pixels。

中文编码标准为什么不断扩展？



中文编码标准为什么不断扩展？



如果字符不在字符集中，系统就无法稳定交换它。

如何给全球字符进行编码？ 第一步要做什么？

A

U+0041

中

U+4E2D

Ѝ

U+05E9

U+1F600

Unicode: 给全球字符分配统一码位

A

U+0041

中

U+4E2D

ψ

U+05E9

U+1F600

Unicode 先回答: “这个字符的全球统一编号是多少?”

注意: 码位不是字节。下一步才是 UTF。

有了 Unicode, 为什么还会有 UTF-8 / UTF-16 / UTF-32?

Unicode 码位

U+4E2D

UTF-8

E4 B8 AD

UTF-16BE

4E 2D

UTF-32BE

00 00 4E 2D

有了 Unicode，为什么还会有 UTF-8 / UTF-16 / UTF-32？

Unicode 码位

U+4E2D

UTF-8

E4 B8 AD

UTF-16BE

4E 2D

UTF-32BE

00 00 4E 2D

Unicode 决定“编号”，UTF 决定“字节”。

同一个码位，可以有不同的字节写法。

UTF-8、UTF-16、UTF-32 的差异，本质是“用多少字节”

字符	UTF-8	UTF-16	UTF-32
A	1 字节 41	2 字节 00 41	4 字节 00 00 00 41
语	3 字节 E8 AF AD	2 字节 8B ED	4 字节 00 00 8B ED
	4 字节 F0 9F 98 80	4 字节 D8 3D DE 00	4 字节 00 01 F6 00

同一个 Unicode 码位，不同 UTF 形式会有不同空间成本。

UTF-8、UTF-16、UTF-32，谁更优秀？

UTF-8

传输与存储
兼容 ASCII

UTF-16

平台历史
兼顾 BMP 字符

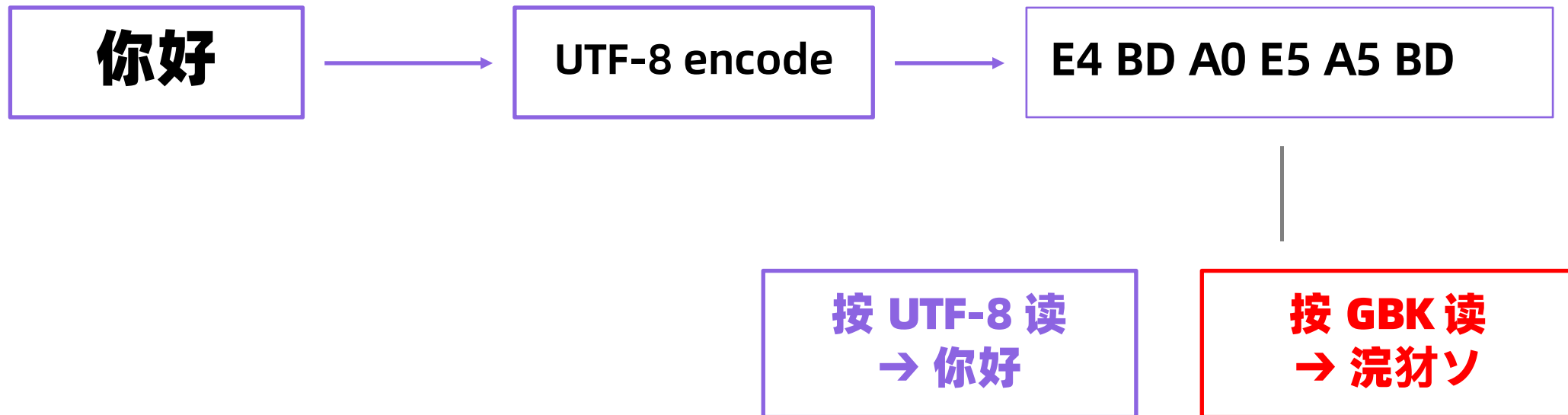
UTF-32

定长简单
空间成本高

没有绝对优秀，只有场景权衡。

同一批 bytes，为什么会变成乱码？

同一批 bytes，为什么会变成乱码？



同一批 bytes，使用不匹配的 decode 规则，就可能显示成乱码。

现在能重新解释整个过程吗？

现在能重新解释整个过程吗？

输入并显示



保存并读取



乱码定位：保存并读取路径中，decode 使用了不匹配的规则。

用三句话解释字符编码

1. 一串 bits 为什么不能自己决定文字意义？

2. Unicode 码位与三种 UTF 各负责什么？

3. bytes 没坏时，为什么仍可能出现乱码？